Evolutionary Computation – Report 3

Local Search Heuristics for TSP

Jędrzej Pluta (160327) **Filip Podalak** (160311)

October 26, 2026

1 Problem Description

The goal of this assignment is to find a cycle that visits exactly half of the available nodes ($k = \lceil n/2 \rceil$), minimizing the total sum of edge distances and selected node costs. Distances are calculated using the Euclidean metric rounded to the nearest integer.

We improve initial solutions using Local Search. To keep the algorithm fast, we evaluate candidate moves by calculating the change in the objective function (Δ) instead of rebuilding and recalculating the entire cycle from scratch. We only use the precomputed distance matrix and cost array.

2 Local Search Design

2.1 Neighborhood Move Types

Following the task requirements, the search neighborhood combines two types of moves:

1. **Inter-route Moves:** Swapping one selected node in the cycle with one unselected node.
2. **Intra-route Moves:** Changing the order of the selected nodes using either:
   - **Two-nodes exchange:** Swapping the positions of two nodes in the cycle.
   - **Two-edges exchange (2-opt):** Reversing the path segment between two nodes to remove crossing edges.

2.2 Mathematical Delta Formulas

Let $p = \pi_{(i-1+k)\%k}$ (previous node), $u = \pi_i$ (current node), and $n = \pi_{(i+1)\%k}$ (next node).

1. Inter-Route Move Delta (replacing selected node u with unselected node v):

$$\Delta_{inter}(i, v) = D[p][v] + D[v][n] - D[p][u] - D[u][n] + c_v - c_u \tag{1}$$

2. Intra-Route Two-Nodes Exchange Delta (swapping nodes at index i and j):

- Adjacent ($j = i + 1$):

$$\Delta_{node}(i, j) = D[\pi_{(i-1+k)\%k}][\pi_j] + D[\pi_i][\pi_{(j+1)\%k}] - D[\pi_{(i-1+k)\%k}][\pi_i] - D[\pi_j][\pi_{(j+1)\%k}] \tag{2}$$

- Non-Adjacent:

$$\begin{aligned}\Delta_{node}(i, j) = {} & D[\pi_{(i-1+k)\%k}][\pi_j] + D[\pi_j][\pi_{(i+1)\%k}] + D[\pi_{(j-1+k)\%k}][\pi_i] + D[\pi_i][\pi_{(j+1)\%k}] \\ & - (D[\pi_{(i-1+k)\%k}][\pi_i] + D[\pi_i][\pi_{(i+1)\%k}] + D[\pi_{(j-1+k)\%k}][\pi_j] + D[\pi_j][\pi_{(j+1)\%k}])\end{aligned} \tag{3}$$

3. Intra-Route Two-Edges Exchange (2-opt) Delta (reversing path between $i+1$ and j):

$$\Delta_{2opt}(i, j) = D[\pi_i][\pi_j] + D[\pi_{i+1}][\pi_{(j+1)\%k}] - D[\pi_i][\pi_{i+1}] - D[\pi_j][\pi_{(j+1)\%k}] \tag{4}$$

2.3 Search Strategies and Randomization

- **Steepest Local Search:** Evaluates the delta for every possible inter-route and intra-route move in the neighborhood. We pick the single move that provides the best improvement (lowest $\Delta < 0$).
- **Greedy Local Search:** To minimize the number of evaluated moves, we do not evaluate the whole neighborhood. Instead, we generate a list of all valid moves, shuffle it randomly using a Mersenne Twister engine, and calculate deltas one by one. We immediately apply the first move that yields $\Delta < 0$ and restart the search.

3 Algorithms Pseudocode

Algorithm 1 Steepest Local Search

```
Input: cycle
loop
    best_delta = 0
    best_move = null
    for each move in Neighborhood(cycle) do
        delta = CalculateDelta(cycle, move)
        if delta < best_delta then
            best_delta = delta
            best_move = move
        end if
    end for
    break if best_delta ≥ 0                    ▷ Stop if no improvement is found
    Apply best_move to cycle
end loop
return cycle
```

Algorithm 2 Greedy Local Search

```
Input: cycle
loop
    moves_list = Shuffle(GenerateAllMoves(cycle))
    improved = false
    for each move in moves_list do
        if CalculateDelta(cycle, move) < 0 then
            Apply move to cycle
            improved = true
            break loop                         ▷ Apply the first good move immediately
        end if
    end for
    break if not improved                      ▷ Stop if stuck
end loop
return cycle
```

4 Computational Experiments

We evaluated 8 combinations of Local Search (steepest/greedy, nodes/edges, random/greedy start). Each method ran 200 times. Greedy starting solutions used each of the 200 nodes as the starting node once.

4.1 Objective Function Results

Table 1: Objective Function Values Across Instances: Average (Min – Max)

Method	**Instance 1 (TSPA.csv)**	**Instance 2 (TSPB.csv)**
Random Solution	263132.65 (232767 – 290020)	207121.19 (184861 – 227890)
Greedy Cycle Baseline	73793.40 (72590 – 75281)	59649.91 (55655 – 63734)
LS Steepest + Nodes (Random Start)	90202.96 (83841 – 97843)	66072.92 (59743 – 73138)
LS Steepest + Edges (Random Start)	76157.96 (73773 – 79690)	**52604.75** (**50578** – 56816)
LS Greedy + Nodes (Random Start)	88055.50 (81061 – 97326)	63740.53 (57870 – 70250)
LS Greedy + Edges (Random Start)	76231.32 (73790 – 78919)	52962.00 (51020 – 55783)
LS Steepest + Nodes (Greedy Start)	73472.41 (72415 – 74736)	59232.55 (55406 – 63516)
LS Steepest + Edges (Greedy Start)	73178.55 (72088 – **74544**)	58666.13 (54818 – 63178)
LS Greedy + Nodes (Greedy Start)	73471.34 (72415 – 74736)	59229.76 (55225 – 63516)
LS Greedy + Edges (Greedy Start)	**73175.78** (**72088** – 74565)	58662.76 (54420 – 63163)

4.2 Execution Times Results

Table 2: Execution Times in Milliseconds: Average (Min – Max)

Method	**Instance 1 Time [ms]**	**Instance 2 Time [ms]**
Random Solution	0.00 (0.00 – 0.01)	0.00 (0.00 – 0.00)
Greedy Cycle Baseline	0.83 (0.79 – 1.10)	0.82 (0.79 – 1.30)
LS Steepest + Nodes (Random Start)	5.00 (3.88 – 7.18)	4.83 (3.63 – 5.93)
LS Steepest + Edges (Random Start)	2.80 (2.38 – 3.87)	2.75 (2.40 – 3.69)
LS Greedy + Nodes (Random Start)	40.95 (32.88 – 71.15)	41.36 (33.32 – 50.66)
LS Greedy + Edges (Random Start)	39.84 (33.43 – 45.62)	41.00 (32.99 – 50.74)
LS Steepest + Nodes (Greedy Start)	**0.94** (**0.83** – 1.21)	1.10 (**0.87** – 1.66)
LS Steepest + Edges (Greedy Start)	0.96 (0.83 – 1.31)	**1.09** (0.92 – 1.64)
LS Greedy + Nodes (Greedy Start)	1.39 (0.94 – 2.46)	1.84 (1.07 – 4.21)
LS Greedy + Edges (Greedy Start)	1.77 (1.07 – 2.83)	2.60 (1.61 – 5.58)

5 2D Visualizations of the Best Solutions

The figures below present the 2D visualisations of the best Hamiltonian cycles found by the Local Search algorithms on both datasets. Node colors indicate the node costs.

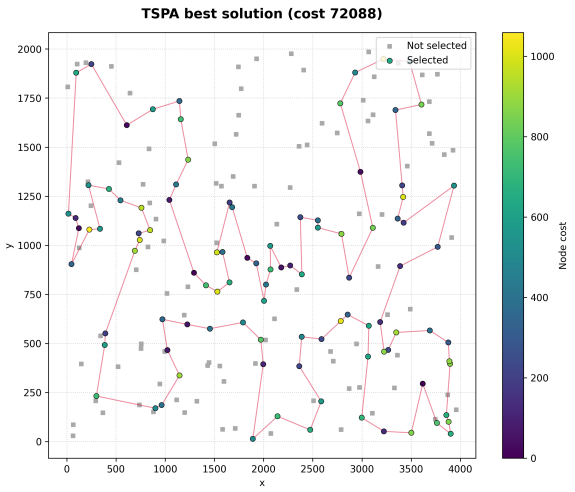


Figure 1: Best solution for `TSPA.csv`

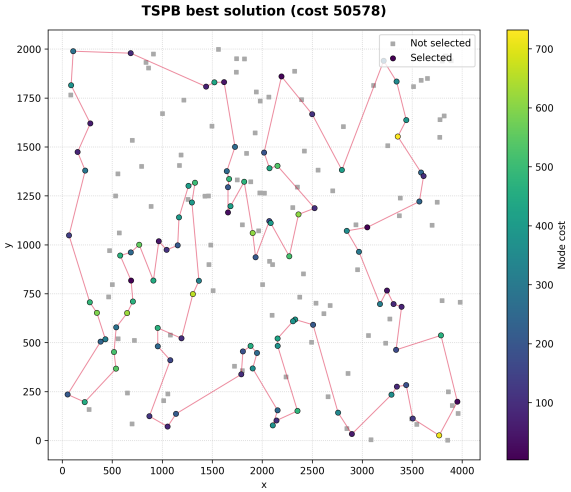


Figure 2: Best solution for `TSPB.csv`

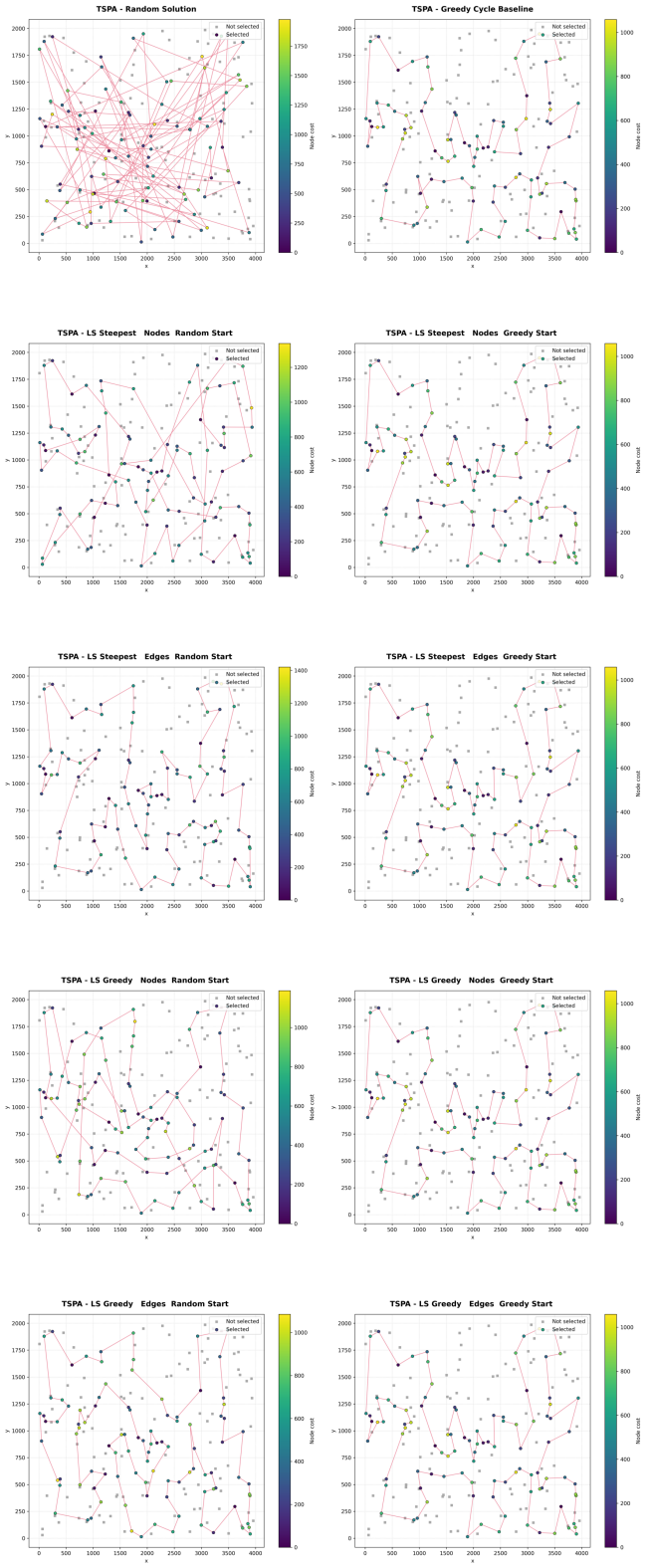


Figure 3: Paired visualizations for all methods on `TSPA.csv`

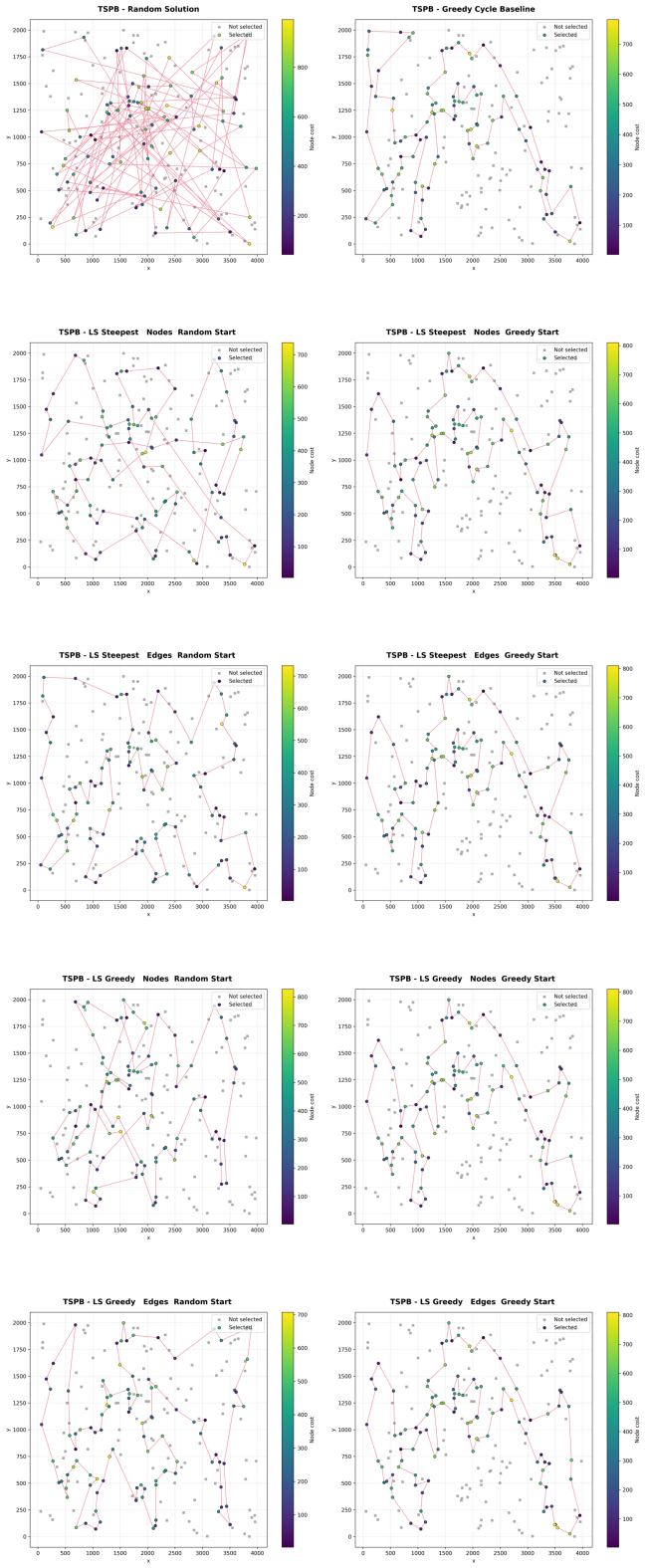


Figure 4: Paired visualizations for all methods on `TSPB.csv`

6 Best Solutions (Node Indices)

The following 0-indexed node lists are the best solutions saved for each method. Each list contains the selected nodes in cycle order.

6.1 Instance 1 (`TSPA.csv`)

Random Solution

```
180 132 100 152 16 69 61 187 3 49 191 10 15 6 121 193 93 29 189 48 143 85 98 181 65 40 182 90
    77 32 92 188 96 62 115 155 194 56 134 82 138 97 162 53 157 192 19 54 74 144 114 196 66 198
    129 120 87 142 59 185 116 161 158 2 30 0 81 106 140 12 13 167 168 179 117 195 52 175 127 51
     28 169 151 130 145 105 71 108 160 33 46 156 8 63 55 41 4 135 123 31
```

Greedy Cycle Baseline

```
31 116 149 168 63 11 49 60 108 152 100 90 123 83 160 97 127 69 171 2 189 30 187 141 87 45 73
    135 114 181 192 158 58 84 134 72 176 150 151 182 57 64 107 88 78 193 153 170 146 159 130
    113 89 42 106 93 47 147 50 3 169 1 178 101 29 163 136 186 164 142 8 174 131 54 9 33 36 177
    52 179 53 140 144 67 16 51 156 35 185 102 28 180 132 76 4 71 157 91 80 48
```

LS Greedy + Edges (Greedy Start)

```
31 116 149 63 168 11 49 60 108 152 100 90 123 83 160 97 127 69 171 2 189 187 30 141 87 45 73
    135 114 181 192 158 58 84 134 72 176 150 151 182 57 64 107 88 78 193 153 170 146 159 130
    113 89 42 106 93 47 147 50 3 169 1 178 101 29 163 136 186 164 85 174 8 54 9 33 131 36 177
    52 179 53 140 144 67 16 51 156 35 185 102 28 180 132 76 4 71 157 91 80 48
```

LS Greedy + Edges (Random Start)

```
58 84 134 72 176 8 174 150 106 89 42 151 182 57 107 88 78 193 153 170 146 159 130 47 93 50 3
    169 1 178 29 136 164 9 54 33 143 139 112 115 157 71 91 80 48 31 60 108 152 49 63 149 116
    168 95 97 127 69 171 2 189 187 141 30 161 160 83 123 100 90 132 76 4 51 16 67 144 140 53
    179 52 177 36 131 41 135 73 45 185 35 156 180 28 87 98 25 114 181 192 158
```

LS Greedy + Nodes (Greedy Start)

```
31 116 149 63 168 11 49 60 108 152 100 90 123 83 160 97 127 69 171 2 189 187 30 141 87 45 73
    135 114 181 192 158 58 84 134 72 176 150 151 182 57 64 107 88 78 193 153 170 146 159 130
    113 89 42 106 93 47 147 50 3 169 1 178 101 29 163 136 186 164 142 8 174 131 54 9 33 36 177
    52 179 53 140 144 67 16 51 156 35 185 102 28 180 132 76 4 71 157 91 80 48
```

LS Greedy + Nodes (Random Start)

```
106 150 58 135 87 141 187 30 149 116 168 95 127 97 160 100 152 48 31 60 63 11 49 108 80 91 71 4
     102 28 180 83 123 90 132 76 156 35 185 144 179 177 53 140 45 73 84 72 176 174 54 33 52 67
    16 51 139 112 5 115 157 38 161 92 171 2 189 98 114 181 192 158 151 42 57 182 138 134 99 131
     36 9 164 186 136 29 178 1 3 50 47 130 159 153 193 78 88 107 89 93
```

LS Steepest + Edges (Greedy Start)

```
31 116 149 63 168 11 49 60 108 152 100 90 123 83 160 97 127 69 171 2 189 187 30 141 87 45 73
    135 114 181 192 158 58 84 134 72 176 150 151 182 57 64 107 88 78 193 153 170 146 159 130
    113 89 42 106 93 47 147 50 3 169 1 178 101 29 163 136 186 164 85 174 8 54 9 33 131 36 177
    52 179 53 140 144 67 16 51 156 35 185 102 28 180 132 76 4 71 157 91 80 48
```

LS Steepest + Edges (Random Start)

```
116 60 31 48 80 91 71 157 115 112 139 51 16 35 185 45 135 73 140 53 144 67 52 179 177 36 131 33
     143 9 54 8 174 85 142 164 178 184 136 163 19 29 101 1 169 3 50 93 47 130 159 146 170 78
    193 153 107 64 57 182 151 42 89 113 43 106 150 176 72 134 84 58 158 192 181 114 98 87 28
    180 141 30 187 189 2 171 69 127 97 123 132 76 100 152 108 49 11 168 63 149
```

LS Steepest + Nodes (Greedy Start)

31 116 149 63 168 11 49 60 108 152 100 90 123 83 160 97 127 69 171 2 189 187 30 141 87 45 73
135 114 181 192 158 58 84 134 72 176 150 151 182 57 64 107 88 78 193 153 170 146 159 130
113 89 42 106 93 47 147 50 3 169 1 178 101 29 163 136 186 164 142 8 174 131 54 9 33 36 177
52 179 53 140 144 67 16 51 156 35 185 102 28 180 132 76 4 71 157 91 80 48

LS Steepest + Nodes (Random Start)

67 144 179 84 134 89 113 130 159 170 153 193 78 88 107 57 182 85 136 186 33 36 177 53 140 73 41
131 54 9 8 174 150 176 112 157 71 4 180 28 87 141 30 161 69 104 129 127 97 171 2 187 156
16 51 91 80 48 31 60 108 152 123 90 132 76 100 63 149 116 168 49 160 185 45 135 114 181 58
192 158 151 106 93 47 37 101 178 184 164 3 50 169 1 147 21 55 29 52 35

6.2 Instance 2 (TSPB.csv)

Random Solution

68 78 70 85 67 88 190 97 108 73 156 56 117 39 131 33 76 123 72 79 151 98 173 87 2 21 91 136 155
135 30 99 90 95 166 111 147 146 114 77 94 129 92 8 19 154 23 176 64 49 121 133 103 75 55
138 26 3 174 160 137 102 126 106 143 0 180 175 35 162 104 96 132 167 40 60 81 10 65 152 51
198 71 18 80 62 140 159 12 196 183 74 109 172 45 83 41 120 168 6

Greedy Cycle Baseline

174 36 197 6 180 115 170 97 140 47 155 71 175 134 45 157 60 141 40 33 64 21 189 168 69 160 172
136 133 120 8 117 72 46 86 184 3 173 122 151 98 111 27 135 76 185 162 192 80 52 182 20 74 2
93 110 118 14 153 90 129 127 54 104 57 89 156 43 48 96 198 121 63 130 191 183 30 176 38
154 26 125 177 138 158 145 149 147 42 116 37 146 13 28 65 178 103 29 132 1

LS Greedy + Edges (Greedy Start)

154 38 176 30 183 191 130 63 121 198 58 96 48 43 156 89 57 104 54 78 10 196 114 148 56 118 110
93 9 2 188 14 20 74 153 182 52 90 129 127 80 192 162 144 185 76 135 27 111 98 151 122 173
184 3 86 46 72 117 8 120 133 136 172 160 69 168 189 21 64 33 47 140 97 170 115 180 155 71
40 60 157 45 134 175 36 197 6 29 132 103 178 65 145 28 149 158 177 125 26

LS Greedy + Edges (Random Start)

155 115 180 6 197 36 29 132 178 65 145 28 149 116 158 177 170 125 26 154 140 47 38 21 176 30
130 183 133 136 172 69 160 117 120 63 121 198 58 96 48 89 57 190 70 81 94 105 61 196 114
148 10 90 14 2 74 20 153 129 127 80 192 54 162 76 86 72 46 3 184 122 151 98 27 66 67 24 34
152 181 179 87 82 51 167 73 59 85 64 33 40 60 157 45 134 175 62 71 141

LS Greedy + Nodes (Greedy Start)

30 183 191 130 63 121 198 58 96 48 43 156 89 57 104 54 78 10 196 114 148 143 90 153 118 110 93
9 2 188 14 20 74 182 52 129 127 80 192 162 144 185 76 135 27 111 98 151 122 173 184 3 86 46
72 117 8 120 133 136 172 160 69 168 189 21 64 33 47 140 97 141 40 60 157 45 134 175 71 155
115 180 6 197 36 29 132 103 178 65 145 28 149 158 125 170 26 154 38 176

LS Greedy + Nodes (Random Start)

90 10 114 196 187 70 190 104 76 185 57 89 72 172 136 133 156 43 48 58 96 30 155 197 29 132 178
177 125 26 170 6 180 115 97 154 140 33 49 59 85 157 142 45 134 175 71 60 40 151 98 111 3
184 86 46 117 8 120 198 121 42 147 116 50 130 191 183 160 69 168 189 21 176 38 47 64 4 73
167 51 82 87 179 181 152 34 24 31 67 66 162 54 192 80 127 129 153 20 14

LS Steepest + Edges (Greedy Start)

158 149 28 65 178 103 132 29 36 197 6 180 115 155 71 175 134 45 157 60 141 40 33 64 21 189 168
69 160 172 136 133 120 8 117 72 46 86 3 184 173 122 151 98 111 27 135 76 185 144 162 192 80
127 129 52 182 153 74 20 14 188 2 9 93 110 118 90 143 148 114 196 10 78 54 104 57 89 156
43 48 96 58 198 121 63 130 191 183 30 176 38 47 140 154 97 170 177 125 26

LS Steepest + Edges (Random Start)

170 115 180 6 197 36 174 1 132 29 103 178 65 145 28 149 146 37 116 121 198 96 120 133 136 172
69 160 117 122 151 184 3 27 135 76 86 46 72 89 57 104 190 70 187 18 196 114 10 54 162 192
80 127 129 90 153 118 110 93 2 14 20 74 171 99 67 24 34 152 181 88 87 82 179 51 167 73 59
85 45 134 175 60 71 155 40 33 64 21 183 30 38 47 140 154 97 26 125 177

LS Steepest + Nodes (Greedy Start)

30 183 191 130 63 121 198 58 96 48 43 156 89 57 104 54 78 10 196 114 148 143 90 153 118 110 93
9 2 188 14 20 74 182 52 129 127 80 192 162 144 185 76 135 27 111 98 151 122 173 184 3 86 46
72 117 8 120 133 136 172 160 69 168 189 21 64 33 47 140 97 141 40 60 157 45 134 175 71 155
115 180 6 197 36 29 132 103 178 65 28 149 158 177 170 125 26 154 38 176

LS Steepest + Nodes (Random Start)

59 51 167 99 39 171 80 192 54 162 104 190 70 187 196 114 10 127 129 90 143 148 78 76 3 184 69
38 140 97 170 115 6 103 178 132 29 197 36 175 134 45 85 82 87 179 67 66 24 34 181 152 21
191 130 147 116 149 28 145 65 158 30 183 133 136 172 160 151 27 74 20 14 2 93 110 153 46 72
117 8 120 121 198 96 89 57 86 173 122 64 47 154 26 125 177 155 71 60 40

7 Verification with Solution Checker

All best solutions presented in Sections 5 and 6 were verified using the solution checker.

8 Source Code

The complete C++ implementation is available at:
`https://github.com/fpodalak/Evolutionary-Computation`

9 Conclusions

1. **Impact of Move Type (2-opt vs. Node Swap):** The two-edges exchange method consistently beat node swaps because uncrossing overlapping paths makes a much bigger difference. With random starts, steepest 2-opt lowered the average cost from 90,202 to 76,157 on TSPA, and from 66,072 to 52,604 on TSPB.

2. **Impact of Starting Solution:** Starting with a Greedy Cycle made things faster and better on TSPA (dropping the cost for steepest 2-opt from 76,157 to 73,178 and time from 2.80 ms to 0.96 ms). However, this didn't hold true for TSPB: random starts actually found a better average cost (52,604) than greedy starts (58,666). This shows that while greedy starts are fast and reliable, random starts can sometimes explore better paths depending on the instance.

3. **Steepest vs. Greedy Local Search:** Steepest search was significantly faster than randomized greedy search. For 2-opt with random starts, steepest took around 2.7–2.8 ms compared to ∼40 ms for greedy search, while achieving roughly the same (or slightly better) final scores. Shuffling and checking the move list randomly added heavy overhead without improving solution quality.

4. **Comparison with the Greedy Cycle Baseline:** Local search doesn't automatically beat the initial greedy baseline. On TSPA, random-start 2-opt actually performed worse than the baseline, whereas greedy-start 2-opt improved upon it. Conversely, on TSPB, random-start 2-opt greatly improved the baseline, while greedy-start stayed close to it. This highlights that initialization and local search work best when evaluated together.

5. **Summary of Findings:** Each test ran 200 times across two 200-node instances, showing that results vary depending on how you start. Overall, 2-opt is clearly the better intra-route move, but the best starting strategy depends heavily on the instance. For TSPA, Greedy Cycle followed by steepest 2-opt worked best; for TSPB, a random start with steepest 2-opt yielded the lowest average cost.